Redis 实现点赞功能场景案例文档

一、场景概述

1.1 业务背景

在社交类、内容类平台（如资讯 APP、短视频平台、社区论坛）中，点赞是核心互动功能之一。用户可对文章、评论、视频等内容进行点赞，平台需实时展示点赞数、用户点赞状态，并支持按点赞数排行等衍生需求。传统数据库（如 MySQL）直接处理高频点赞操作易引发性能瓶颈，因此采用 Redis 作为缓存层，利用其高性能、原子性操作特性实现点赞功能。

1.2 核心需求

表格

功能点	需求描述
点赞 / 取消点赞	用户对指定内容（如文章）点赞，重复点赞无效；取消点赞仅对已点赞用户生效
点赞状态查询	快速判断用户是否已对某内容点赞
点赞数统计	实时展示单条内容的总点赞数
点赞用户列表	按点赞时间排序展示点赞用户（分页）
高并发支持	保障高流量场景下点赞数准确、无并发问题

二、技术选型与设计思路

2.1 技术栈

- 缓存中间件：Redis（6.x+）
- 开发语言：Java（Spring Boot 2.x）/ Python（可选）
- Redis 客户端：Spring Data Redis（Java）、redis-py（Python）

2.2 Redis 数据结构设计

基于 Redis 不同数据结构的特性，针对性设计 Key-Value 结构：

表格

功能场景	数据结构	Key 命名规范	Value 说明	核心操作
点赞状态存储	Hash	`like:status:{businessId}`	Field = 用户 ID，Value=1（已点赞）/0（取消点赞）	hget（查状态）、hset（改状态）
点赞数统计	String	`like:count:{businessId}`	存储内容的总点赞数（数值型）	incr（点赞 + 1）、decr（取消 - 1）
点赞用户列表 + 时间	ZSet	`like:users:{businessId}`	Member = 用户 ID，Score = 点赞时间戳	zadd（添加用户）、zrevrange（按时间倒序查列表）

说明：`businessId` 为业务唯一标识（如文章 ID、视频 ID、评论 ID），通过前缀 + 业务 ID 避免 Redis Key 冲突。

2.3 核心逻辑流程

2.3.1 点赞流程

```
flowchart TD
    A[用户发起点赞请求] --> B[拼接Redis Key（status/count/users）]
    B --> C[查询Hash中用户点赞状态]
    C -->|已点赞（Value=1）| D[返回“重复点赞”，流程结束]
    C -->|未点赞（无值/Value=0）| E[原子操作：Hash设为1 + String计数+1 + ZSet添加用户+时间戳]
    E --> F[设置Key过期时间（可选）]
    F --> G[返回“点赞成功”]
```

2.3.2 取消点赞流程

```
flowchart TD
    A[用户发起取消点赞请求] --> B[拼接Redis Key（status/count/users）]
    B --> C[查询Hash中用户点赞状态]
    C -->|未点赞（无值/Value=0）| D[返回“未点赞，无需取消”，流程结束]
    C -->|已点赞（Value=1）| E[原子操作：Hash设为0 + String计数-1 + ZSet移除用户]
    E --> F[返回“取消点赞成功”]
```

三、代码实现（Java + Spring Boot）

3.1 环境依赖

```
<!-- pom.xml 引入Redis依赖 -->
<dependency>
    <groupId>org.springframework.boot</groupId>
    <artifactId>spring-boot-starter-data-redis</artifactId>
</dependency>
<!-- 可选：引入Lombok简化代码 -->
<dependency>
    <groupId>org.projectlombok</groupId>
    <artifactId>lombok</artifactId>
    <optional>true</optional>
</dependency>
```

3.2 核心配置（RedisTemplate）

```
import org.springframework.context.annotation.Bean;
import org.springframework.context.annotation.Configuration;
import org.springframework.data.redis.connection.RedisConnectionFactory;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.data.redis.serializer.GenericJackson2JsonRedisSerializer;
import org.springframework.data.redis.serializer.StringRedisSerializer;

@Configuration
```

```
public class RedisConfig {
    @Bean
    public RedisTemplate<String, Object> redisTemplate(RedisConnectionFactory factory)
{
        RedisTemplate<String, Object> template = new RedisTemplate<>();
        template.setConnectionFactory(factory);
        // Key序列化：String
        template.setKeySerializer(new StringRedisSerializer());
        template.setHashKeySerializer(new StringRedisSerializer());
        // Value序列化：JSON
        template.setValueSerializer(new GenericJackson2JsonRedisSerializer());
        template.setHashValueSerializer(new GenericJackson2JsonRedisSerializer());
        template.afterPropertiesSet();
        return template;
    }
}
```

3.3 点赞服务实现

```
import lombok.RequiredArgsConstructor;
import org.springframework.data.redis.core.RedisTemplate;
import org.springframework.stereotype.Service;

import java.util.Set;
import java.util.concurrent.TimeUnit;

/**
 * 点赞功能核心服务类
 */
@Service
@RequiredArgsConstructor
public class LikeService {
    // Redis Key前缀常量
    private static final String LIKE_STATUS_KEY = "like:status:%s";   // 点赞状态Key
    private static final String LIKE_COUNT_KEY = "like:count:%s";     // 点赞数Key
    private static final String LIKE_USERS_KEY = "like:users:%s";     // 点赞用户列表
Key
```

```java
private final RedisTemplate<String, Object> redisTemplate;

/**
 * 点赞操作
 * @param businessId 业务ID（如文章ID）
 * @param userId     用户ID
 * @return 操作结果：true=点赞成功，false=重复点赞
 */
public boolean like(String businessId, String userId) {
    // 1. 拼接Key
    String statusKey = String.format(LIKE_STATUS_KEY, businessId);
    String countKey = String.format(LIKE_COUNT_KEY, businessId);
    String usersKey = String.format(LIKE_USERS_KEY, businessId);

    // 2. 检查是否已点赞
    Object isLiked = redisTemplate.opsForHash().get(statusKey, userId);
    if (isLiked != null && "1".equals(isLiked.toString())) {
        return false;
    }

    // 3. 原子操作：更新状态、计数、用户列表
    redisTemplate.opsForHash().put(statusKey, userId, "1");
    redisTemplate.opsForValue().increment(countKey, 1);
    redisTemplate.opsForZSet().add(usersKey, userId, System.currentTimeMillis());

    // 4. 可选：设置Key过期时间（7天，避免Redis内存膨胀）
    redisTemplate.expire(statusKey, 7, TimeUnit.DAYS);
    redisTemplate.expire(countKey, 7, TimeUnit.DAYS);
    redisTemplate.expire(usersKey, 7, TimeUnit.DAYS);

    return true;
}

/**
 * 取消点赞操作
 * @param businessId 业务ID
 * @param userId     用户ID
 * @return 操作结果：true=取消成功，false=未点赞
```

```java
     */
    public boolean cancelLike(String businessId, String userId) {
        String statusKey = String.format(LIKE_STATUS_KEY, businessId);
        String countKey = String.format(LIKE_COUNT_KEY, businessId);
        String usersKey = String.format(LIKE_USERS_KEY, businessId);

        // 检查是否已点赞
        Object isLiked = redisTemplate.opsForHash().get(statusKey, userId);
        if (isLiked == null || !"1".equals(isLiked.toString())) {
            return false;
        }

        // 原子操作：更新状态、计数、用户列表
        redisTemplate.opsForHash().put(statusKey, userId, "0");
        redisTemplate.opsForValue().decrement(countKey, 1);
        redisTemplate.opsForZSet().remove(usersKey, userId);

        return true;
    }

    /**
     * 查询用户点赞状态
     * @param businessId 业务ID
     * @param userId     用户ID
     * @return true=已点赞，false=未点赞
     */
    public boolean isLiked(String businessId, String userId) {
        String statusKey = String.format(LIKE_STATUS_KEY, businessId);
        Object isLiked = redisTemplate.opsForHash().get(statusKey, userId);
        return isLiked != null && "1".equals(isLiked.toString());
    }

    /**
     * 查询内容总点赞数
     * @param businessId 业务ID
     * @return 点赞数（默认0）
     */
    public long getLikeCount(String businessId) {
```

```
        String countKey = String.format(LIKE_COUNT_KEY, businessId);
        Object count = redisTemplate.opsForValue().get(countKey);
        return count == null ? 0 : Long.parseLong(count.toString());
    }

    /**
     * 分页查询点赞用户列表（按点赞时间倒序）
     * @param businessId 业务ID
     * @param pageNum    页码（从1开始）
     * @param pageSize   页大小
     * @return 用户ID列表
     */
    public Set<Object> getLikeUserList(String businessId, int pageNum, int pageSize) {
        String usersKey = String.format(LIKE_USERS_KEY, businessId);
        long start = (pageNum - 1) * pageSize;
        long end = pageNum * pageSize - 1;
        return redisTemplate.opsForZSet().reverseRange(usersKey, start, end);
    }
}
```

3.4 接口层（Controller）

```
import lombok.RequiredArgsConstructor;
import org.springframework.web.bind.annotation.*;

import java.util.Set;

@RestController
@RequestMapping("/api/like")
@RequiredArgsConstructor
public class LikeController {
    private final LikeService likeService;

    /**
     * 点赞接口
     */
    @PostMapping("/{businessId}/{userId}")
    public String like(@PathVariable String businessId, @PathVariable String userId) {
```

```
        boolean success = likeService.like(businessId, userId);
        return success ? "点赞成功" : "重复点赞，操作无效";
    }

    /**
     * 取消点赞接口
     */
    @DeleteMapping("/{businessId}/{userId}")
    public String cancelLike(@PathVariable String businessId, @PathVariable String
userId) {
        boolean success = likeService.cancelLike(businessId, userId);
        return success ? "取消点赞成功" : "未点赞，无需取消";
    }

    /**
     * 查询点赞状态
     */
    @GetMapping("/status/{businessId}/{userId}")
    public Boolean isLiked(@PathVariable String businessId, @PathVariable String
userId) {
        return likeService.isLiked(businessId, userId);
    }

    /**
     * 查询点赞数
     */
    @GetMapping("/count/{businessId}")
    public Long getLikeCount(@PathVariable String businessId) {
        return likeService.getLikeCount(businessId);
    }

    /**
     * 查询点赞用户列表
     */
    @GetMapping("/users/{businessId}")
    public Set<Object> getLikeUserList(
            @PathVariable String businessId,
            @RequestParam(defaultValue = "1") int pageNum,
```

```
            @RequestParam(defaultValue = "10") int pageSize) {
        return likeService.getLikeUserList(businessId, pageNum, pageSize);
    }
}
```

四、功能测试

4.1 测试用例

表格

测试场景	操作步骤	预期结果
首次点赞	调用 `/api/like/1001/888`（文章 1001，用户 888）	返回 “点赞成功”；点赞数变为 1；用户 888 状态为已点赞
重复点赞	再次调用 `/api/like/1001/888`	返回 “重复点赞，操作无效”；点赞数仍为 1
取消点赞	调用 `/api/like/1001/888`（DELETE）	返回 “取消点赞成功”；点赞数变为 0；用户 888 状态为未点赞
查询点赞数	调用 `/api/like/count/1001`	取消后返回 0，点赞后返回 1
查询点赞用户	调用 `/api/like/users/1001?pageNum=1&pageSize=10`	点赞后返回包含 888 的列表，取消后列表为空

4.2 测试工具

- 接口测试：Postman、Swagger
- Redis 数据验证：Redis Desktop Manager、 `redis-cli` 命令（如 `hget like:status:1001 888` 、 `get like:count:1001` ）

五、性能与扩展优化

5.1 高并发优化

- 原子性保障：Redis 单命令（incr/decr/hset）天然原子性，避免并发下计数错误；
- 管道（Pipeline）：批量操作时使用 Pipeline 减少网络 IO；

- 分布式锁：极端场景（如秒杀内容点赞）可加分布式锁防止超量点赞。

5.2 数据持久化

- Redis 层面：开启 RDB+AOF 混合持久化，避免 Redis 宕机数据丢失；
- 业务层面：异步同步到 MySQL（如定时任务、消息队列），保障点赞数据永久存储。

5.3 功能扩展

- 点赞排行：基于 ZSet 存储 “内容 ID - 点赞数”，通过 `zrevrange` 实现按点赞数排行；
- 点赞通知：点赞后触发消息推送，通知内容作者；
- 限流控制：对单用户点赞频率限流，防止恶意刷赞。

六、注意事项

1. Key 命名规范：统一前缀 + 业务 ID，避免不同业务 Key 冲突；
2. 过期时间：非永久存储的点赞数据需设置过期时间，降低 Redis 内存占用；
3. 空值处理：查询点赞数时，若 Key 不存在需返回 0，避免空指针异常；
4. 数据一致性：Redis 与数据库同步时，需处理 “先更缓存还是先更库” 的问题，建议采用 “缓存更新 + 延迟双删” 方案。